UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment No-3

Student Name: HARSH
UID: 24MCA20045
Branch: MCA
Section/Group: 24MCA-1(B)
Semester: 3rd
Date of Performance: 16/09/2025
Subject Name: Machine Learning Lab
Subject Code: 24CAP-702

Aim: Use MongoDB for performing CRUD operations on Faculty Management System.

Task to be done:

The objective of using Mongoose to connect to a MongoDB database and perform CRUD operations is to:

1. Establish a connection between a Node.js application and a MongoDB database.
2. Provide a structured and schema-based way to define data models.
3. Perform CRUD operations (Create, Read, Update, Delete) on MongoDB data easily and efficiently.
4. Simplify database interactions by using high-level JavaScript methods instead of raw MongoDB queries.
5. Ensure data validation and consistency using Mongoose's built-in schema validation.

Flowchart:

Step 1: Install and Import Mongoose
Step 2: Connect to MongoDB
Step 3: Define a Schema
Step 4: Create a Model
Step 5: Perform CRUD Operations
A. Create (Insert Data)
B. Read (Fetch Data)
C. Update (Modify Existing Data)
D. Delete (Remove Data)
Step 6: Handle Errors and Close Connection

Code:

models/personal.js:

```
const mongoose = require('mongoose');

const personalSchema = new mongoose.Schema({

  employeeId: {
    type: String,
    required: true,
    unique: true
```

```
  },
  firstName: {
    type: String,
    required: true
  },
  lastName: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  phone: String
});
module.exports = mongoose.model('PersonalDetail', personalSchema);
```

models/professional.js:

```
const mongoose = require('mongoose');

const professionalSchema = new mongoose.Schema({
  personalId: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: 'PersonalDetail'
  },
  employeeId: {
    type: String,
    required: true,
    unique: true
  },
  department: String,
  jobTitle: String,
  dateOfJoining: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('ProfessionalDetail', professionalSchema);
```

app.js:

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
```

```
// Routes
const personalRoutes = require('./routes/personalRoutes');
const professionalRoutes = require('./routes/professionalRoutes');

const app = express();
const PORT = 5000;

// Middleware
app.use(bodyParser.json());

// MongoDB connection
mongoose.connect('mongodb://localhost:27017/employee_linked_db', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('MongoDB connection error:', err));

// Routes
app.use('/personal', personalRoutes);
app.use('/professional', professionalRoutes);

// Start Server
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Routes/personalRoutes.js

```
const express = require('express');
const router = express.Router();
const Personal = require('../models/personal');
const Professional = require('../models/professional');

// CREATE Personal Record
router.post('/', async (req, res) => {
    try {
        const newPersonal = new Personal(req.body);
        const savedPersonal = await newPersonal.save();
        res.status(201).json(savedPersonal);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});

// READ Personal Record by employeeId
```

```
router.get('/:employeeId', async (req, res) => {
  try {
    const personal = await Personal.findOne({ employeeId: req.params.employeeId });
    if (!personal) return res.status(404).json({ message: "Not found" });
    res.json(personal);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// HEAD - Check if record exists
router.head('/:employeeId', async (req, res) => {
  try {
    const personal = await Personal.findOne({ employeeId: req.params.employeeId });
    if (!personal) return res.status(404).end();
    res.status(200).end();
  } catch (err) {
    res.status(500).end();
  }
});

// UPDATE (Full) Personal Record by employeeId
router.put('/:employeeId', async (req, res) => {
  try {
    const updated = await Personal.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      req.body,
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// PATCH (Partial Update)
router.patch('/:employeeId', async (req, res) => {
  try {
    const updated = await Personal.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      { $set: req.body },
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
```

```
        res.status(400).json({ message: err.message });
    }
});

// DELETE Personal Record (and linked professional)
router.delete('/:employeeId', async (req, res) => {
    try {
        const personalDoc = await Personal.findOneAndDelete({ employeeId: req.params.employeeId });
        if (!personalDoc) return res.status(404).json({ message: "Not found" });

        await Professional.deleteOne({ personalId: personalDoc._id });
        res.json({ message: "Personal and linked professional records deleted" });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

// OPTIONS
router.options('/', (req, res) => {
    res.set('Allow', 'GET,HEAD,POST,PUT,PATCH,DELETE,OPTIONS');
    res.sendStatus(200);
});

module.exports = router;
```

Routes/professionalRoutes.js

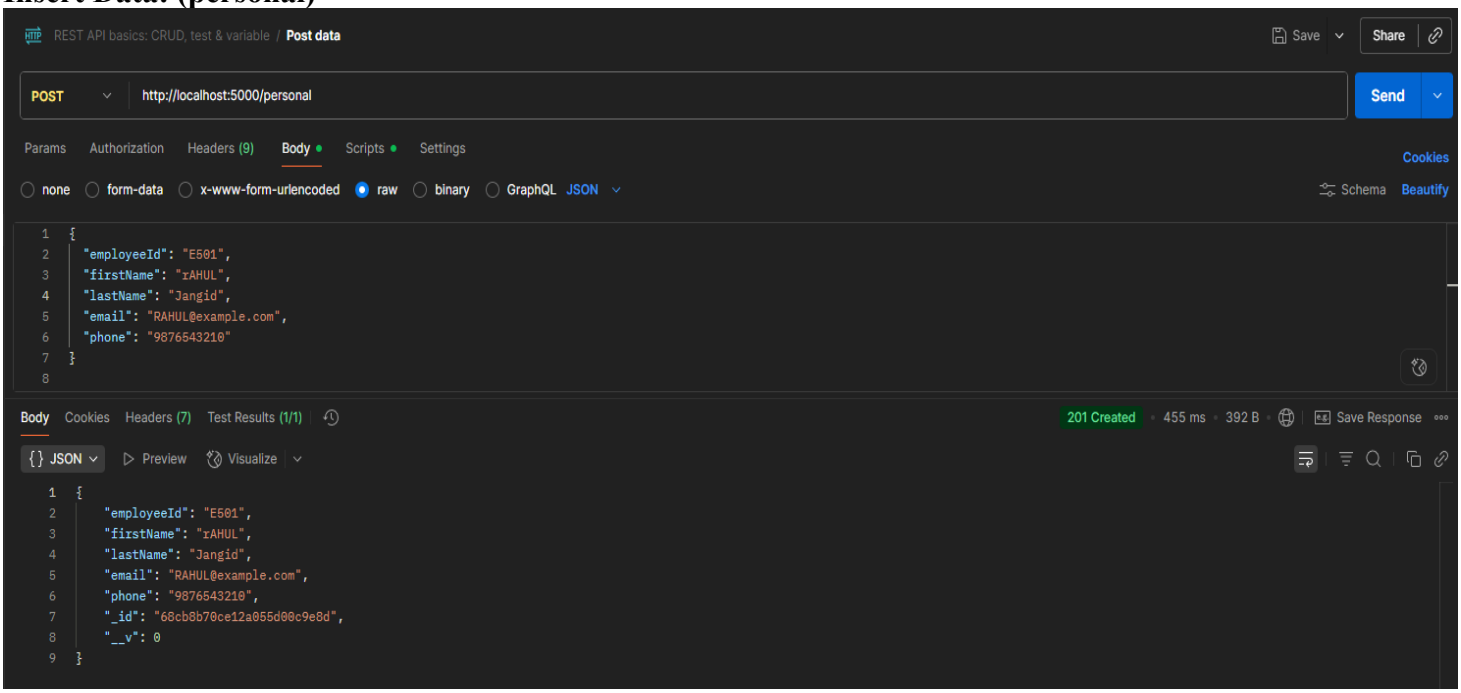
```
const express = require('express');
const router = express.Router();
const Personal = require('../models/personal');
const Professional = require('../models/professional');

// CREATE Professional Record
router.post('/', async (req, res) => {
    try {
        const personalDoc = await Personal.findOne({ employeeId: req.body.employeeId });
        if (!personalDoc) return res.status(404).json({ message: "Personal record not found. Create it first." });

        const newProfessional = new Professional({
            ...req.body,
            personalId: personalDoc._id
        });
        const savedProfessional = await newProfessional.save();
        res.status(201).json(savedProfessional);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
```

```
});

// READ Professional Record (with personal details) by employeeId
router.get('/:employeeId', async (req, res) => {
  try {
    const professional = await Professional.findOne({ employeeId: req.params.employeeId
}).populate('personalId');
    if (!professional) return res.status(404).json({ message: "Not found" });
    res.json(professional);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

// HEAD - Check if record exists
router.head('/:employeeId', async (req, res) => {
  try {
    const professional = await Professional.findOne({ employeeId: req.params.employeeId });
    if (!professional) return res.status(404).end();
    res.status(200).end();
  } catch (err) {
    res.status(500).end();
  }
});

// UPDATE (Full) Professional Record
router.put('/:employeeId', async (req, res) => {
  try {
    const updated = await Professional.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      req.body,
      { new: true }
    );
    if (!updated) return res.status(404).json({ message: "Not found" });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

// PATCH (Partial Update)
router.patch('/:employeeId', async (req, res) => {
  try {
    const updated = await Professional.findOneAndUpdate(
      { employeeId: req.params.employeeId },
      { $set: req.body },
      { new: true }
```

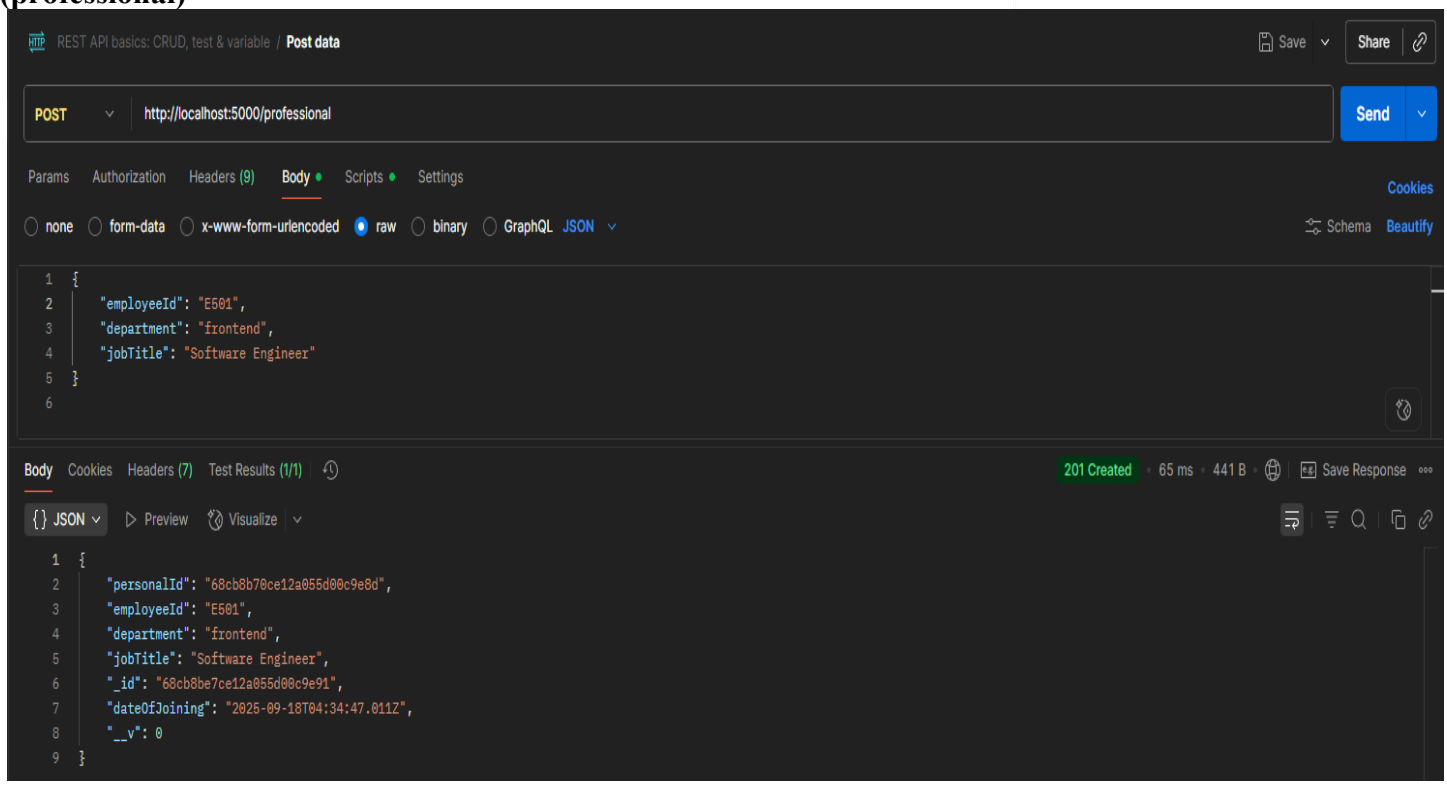
```
        );
        if (!updated) return res.status(404).json({ message: "Not found" });
        res.json(updated);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});

// DELETE Professional Record
router.delete('/:employeeId', async (req, res) => {
    try {
        const deleted = await Professional.findOneAndDelete({ employeeId: req.params.employeeId });
        if (!deleted) return res.status(404).json({ message: "Not found" });
        res.json({ message: "Professional record deleted" });
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});

// OPTIONS
router.options('/', (req, res) => {
    res.set('Allow', 'GET,HEAD,POST,PUT,PATCH,DELETE,OPTIONS');
    res.sendStatus(200);
});

module.exports = router;
```

Output:

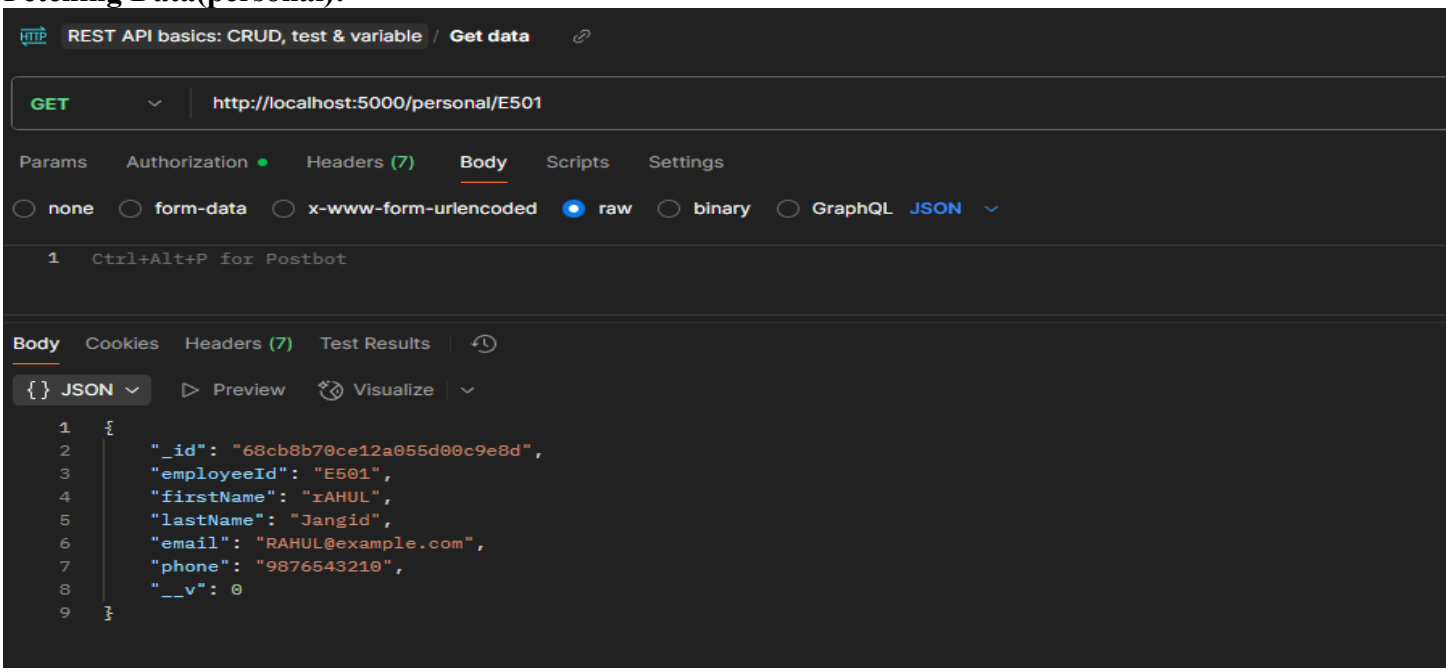
Insert Data: (personal)



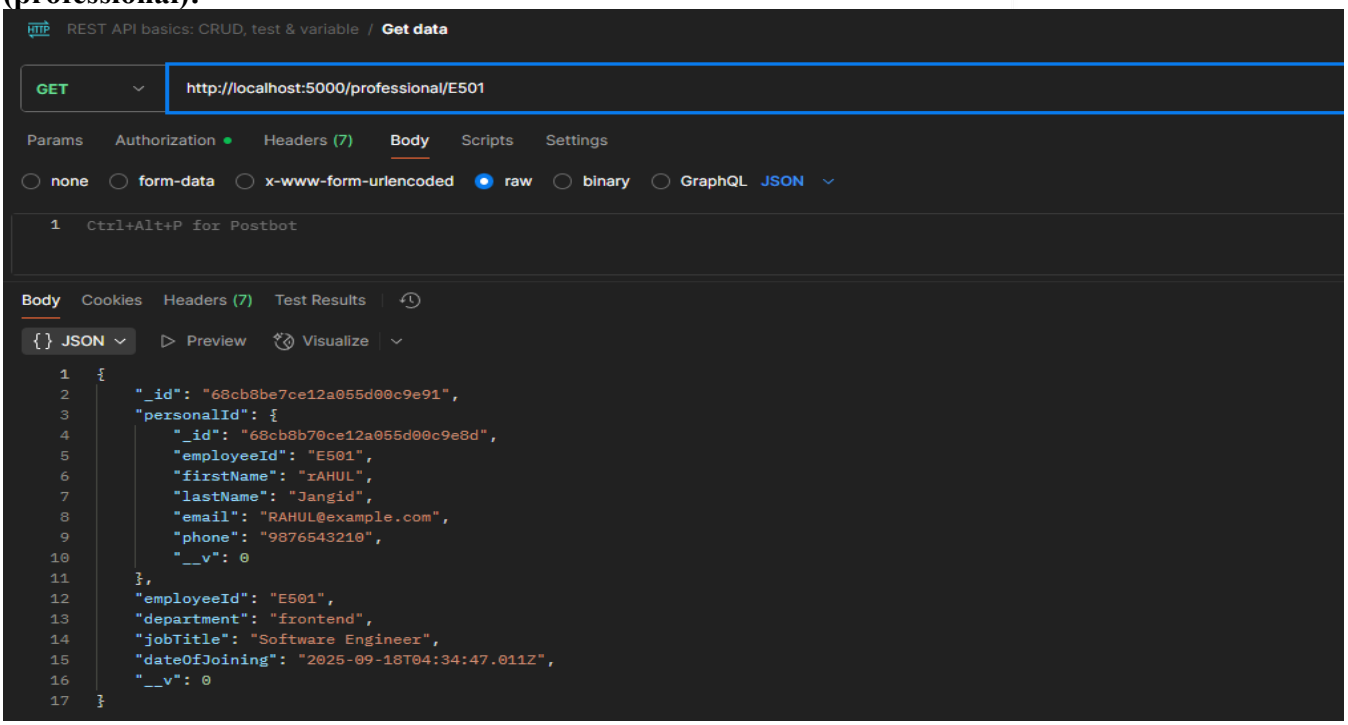
(professional)



Fetching Data(personal):

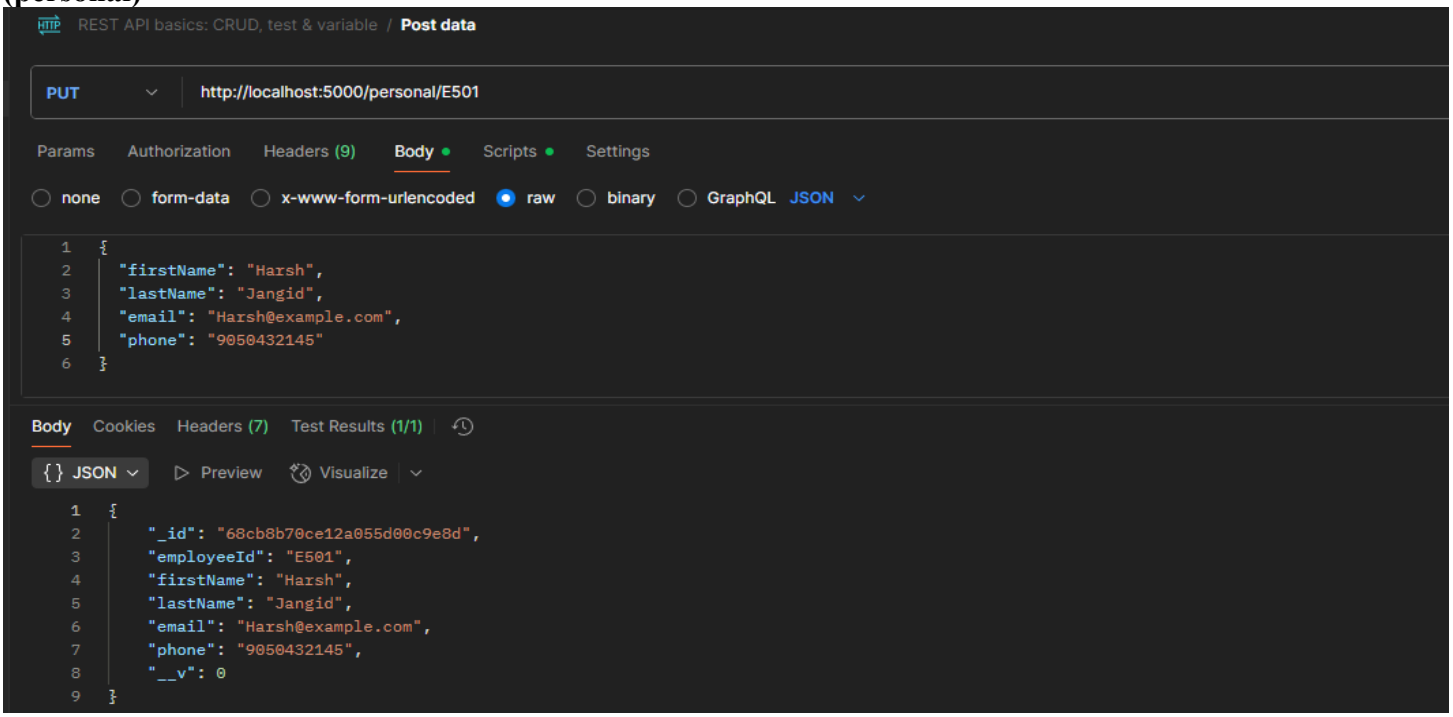


(professional):

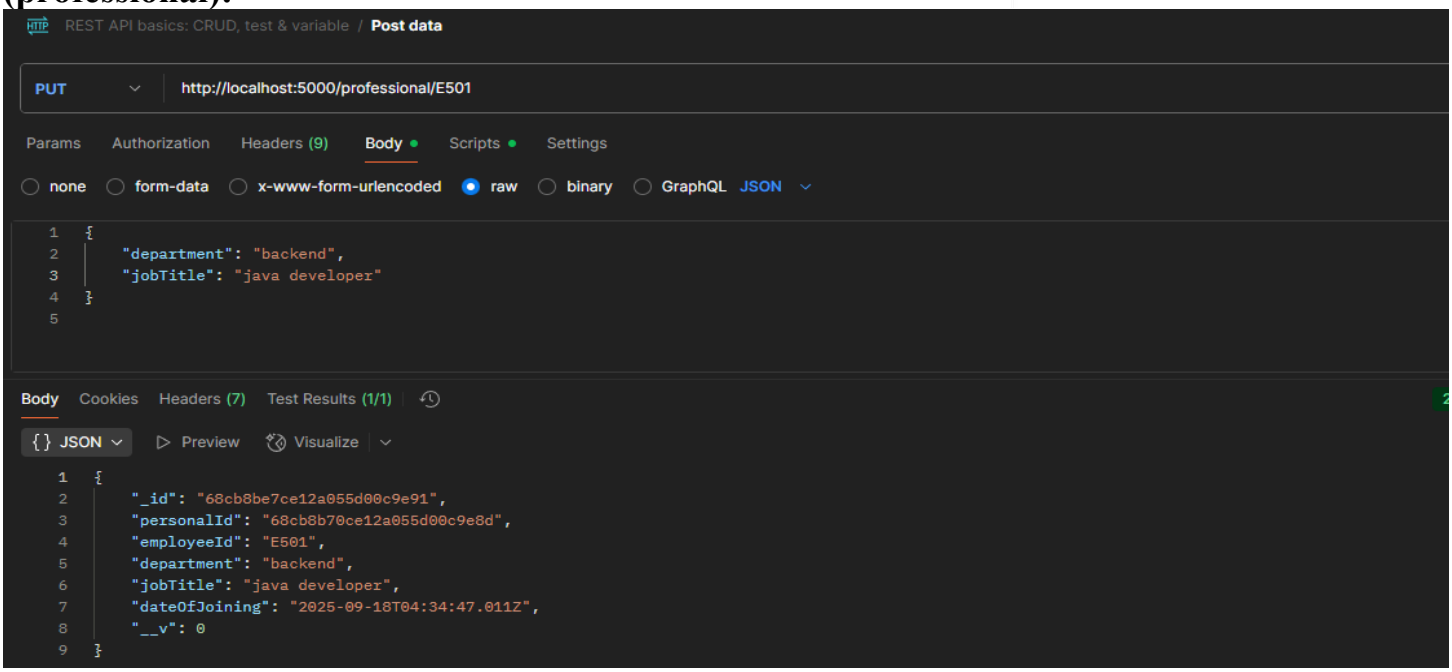


Modifying Data by Id with PUT Request:

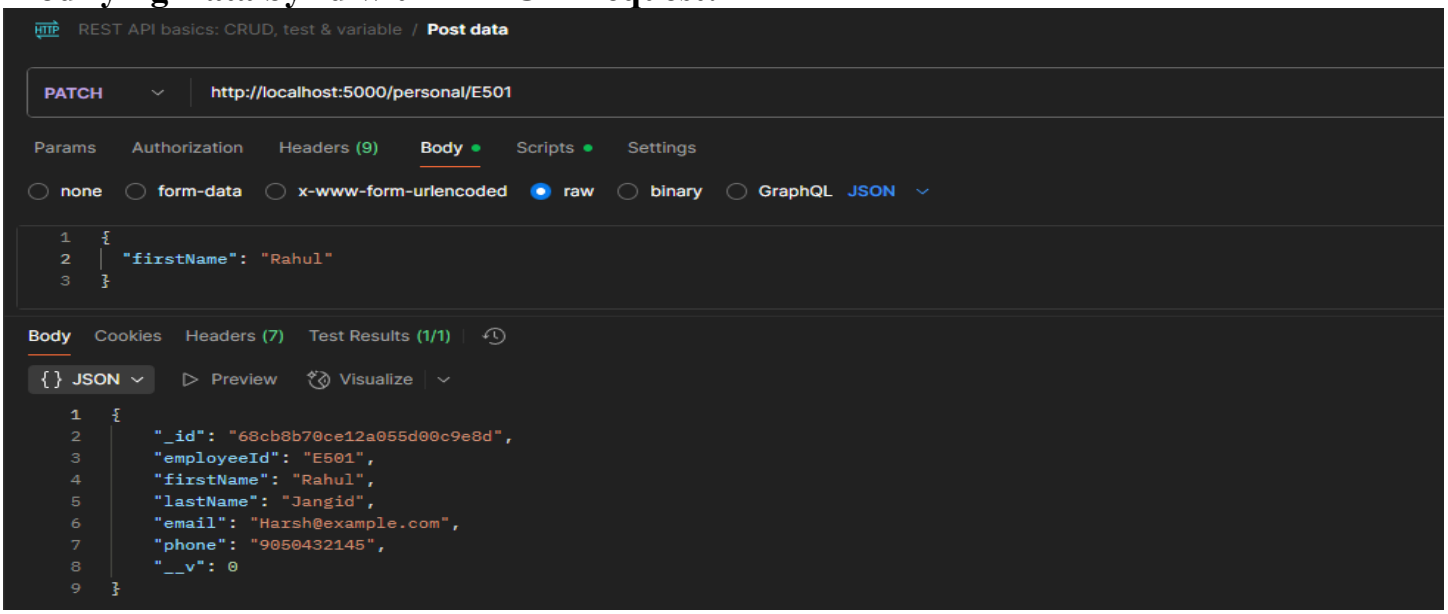
(personal)



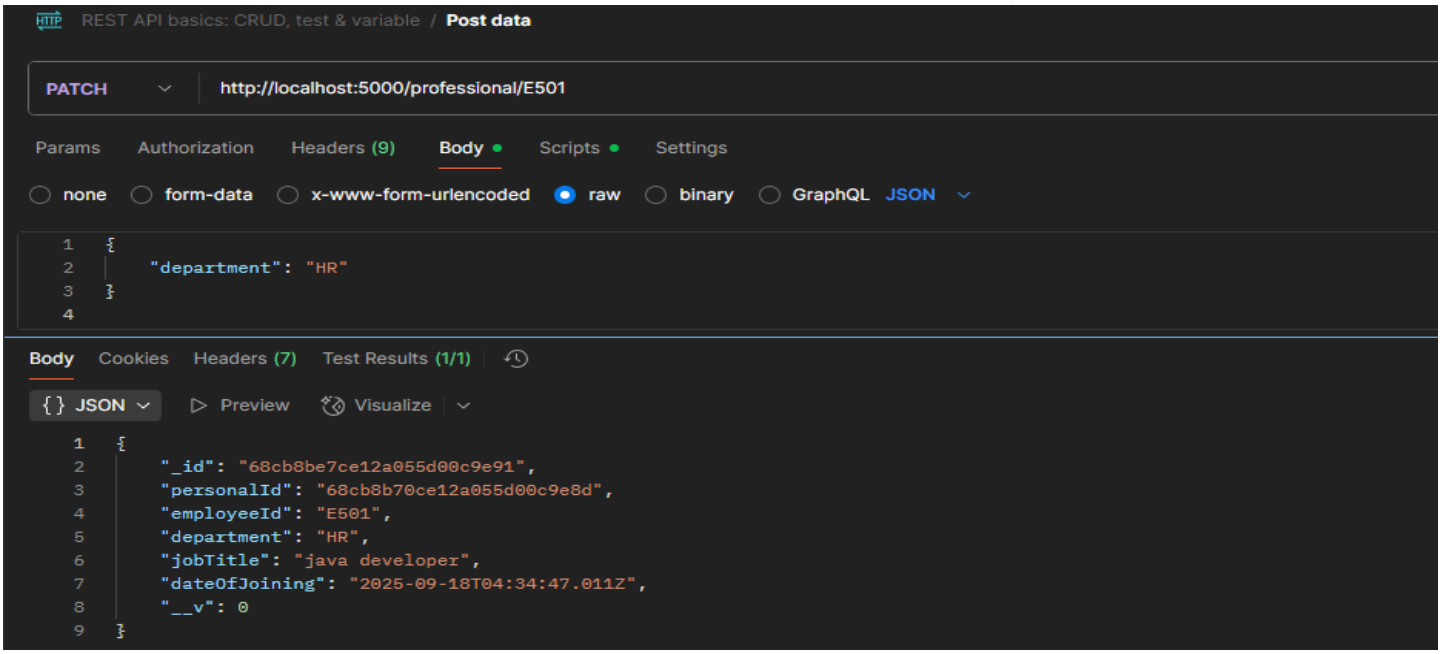
(professional):



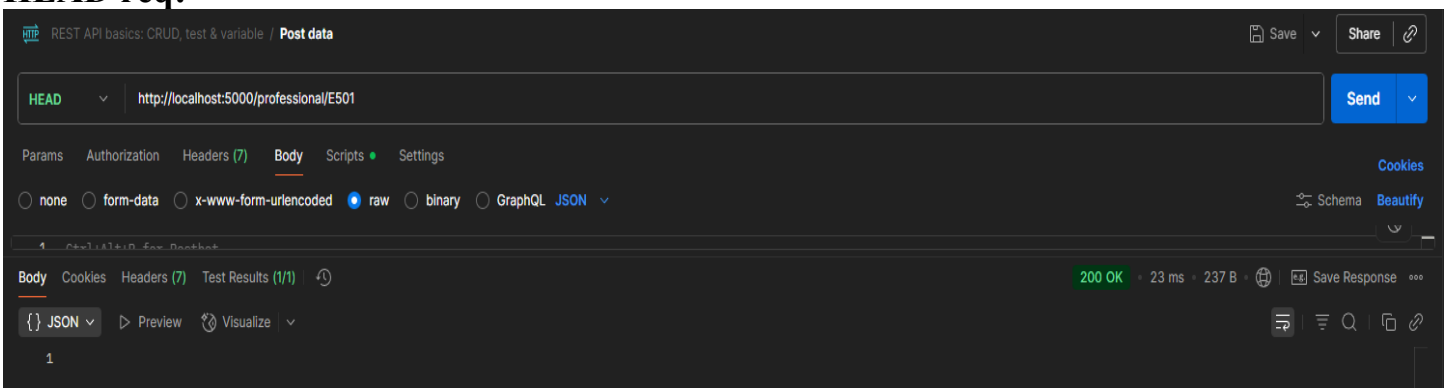
Modifying Data by Id with PATCH Request:



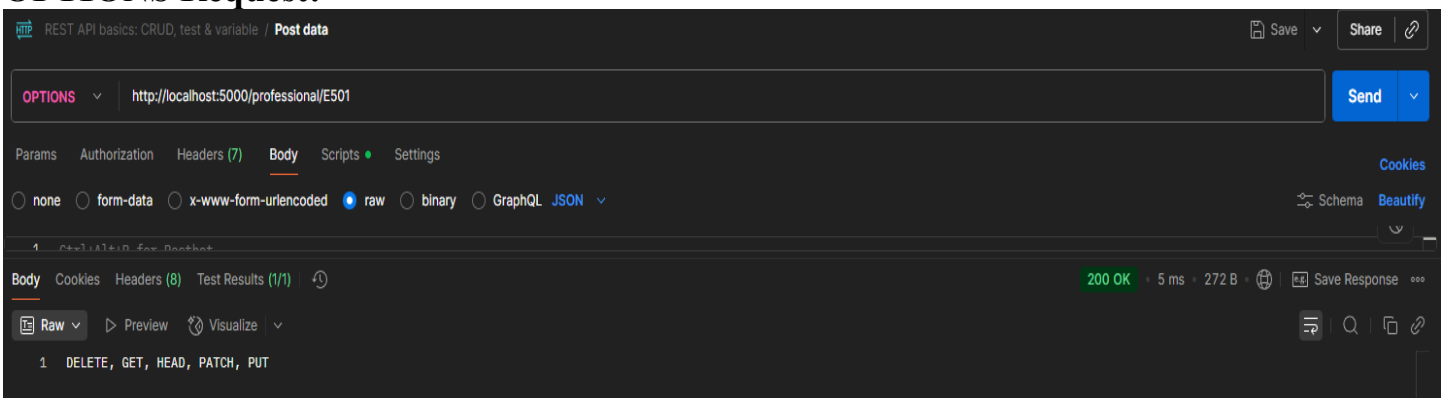
Professional:



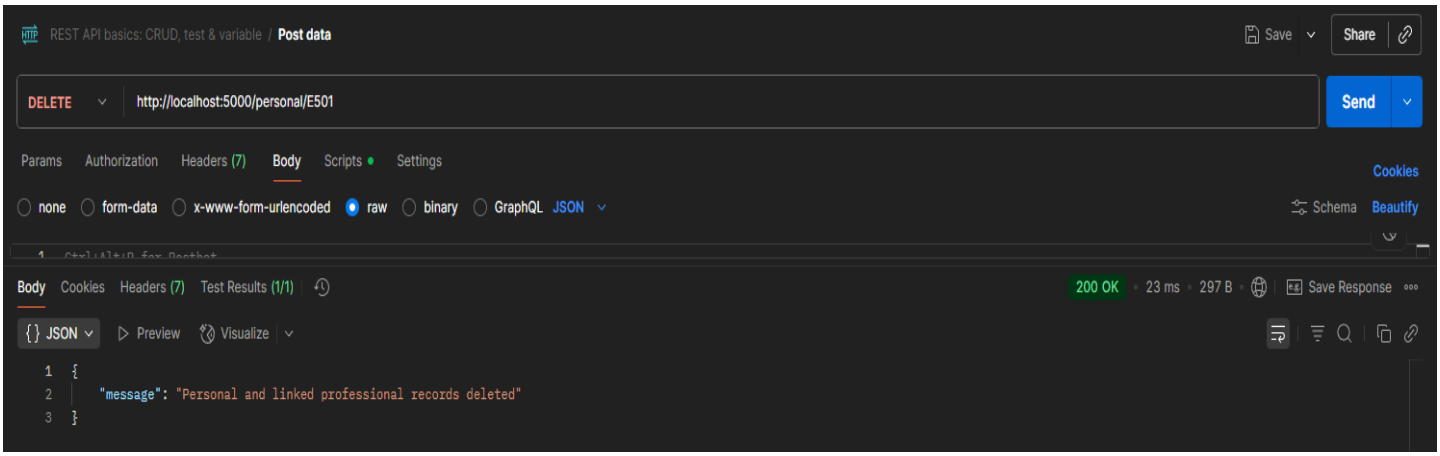
HEAD req:



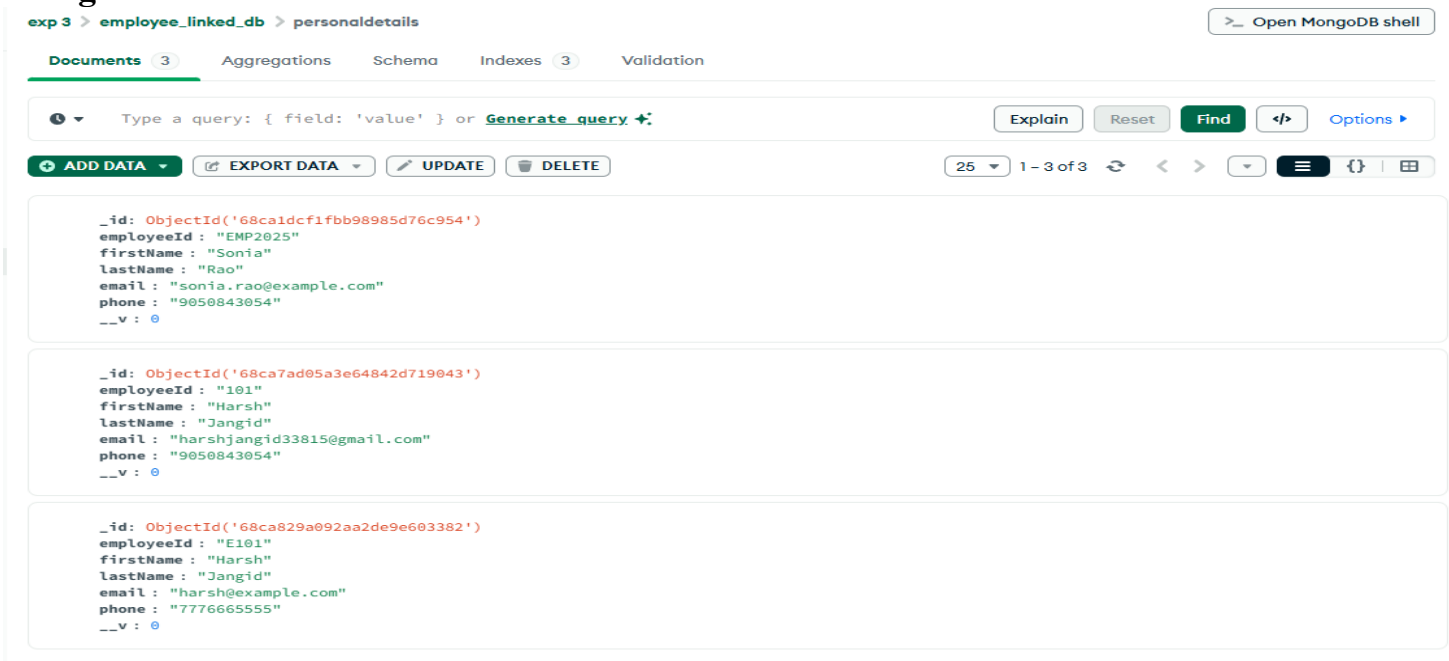
OPTIONS Request:

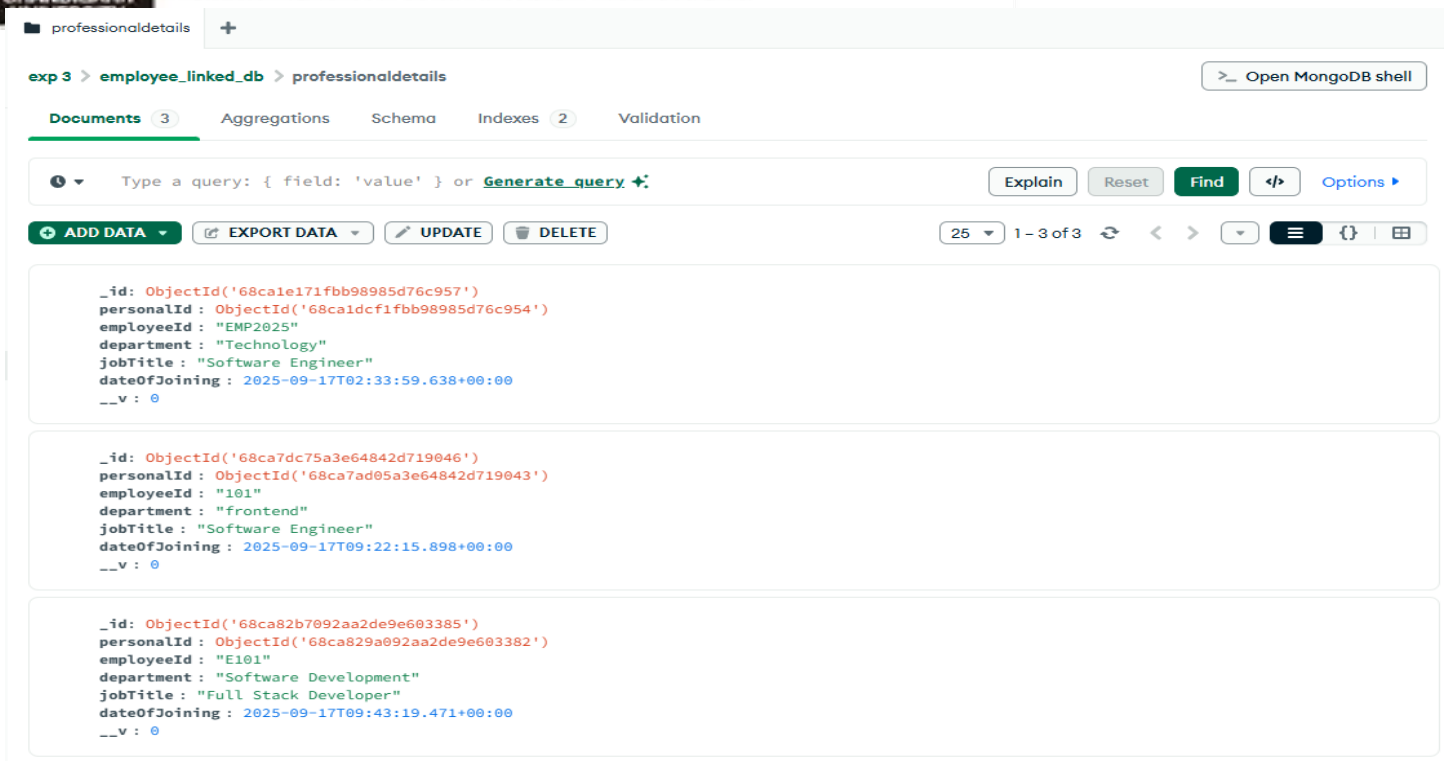


DELETE:



MongoDB data:





Learning Outcomes:

1. Learned to establish a database connection using Mongoose.
2. Designed schemas and models for MongoDB collections.
3. Learned to implement CRUD operations in a Node.js application.
4. Handled proper error handling and response management.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Via		10